

Federated Graph Anomaly Detection via Contrastive Self-Supervised Learning

Xiangjie Kong^{ID}, Senior Member, IEEE, Wenyi Zhang^{ID}, Hui Wang^{ID}, Mingliang Hou^{ID}, Xin Chen^{ID}, Xiaoran Yan^{ID}, Member, IEEE, and Sajal K. Das^{ID}, Fellow, IEEE

Abstract—Attribute graph anomaly detection aims to identify nodes that significantly deviate from the majority of normal nodes, and has received increasing attention due to the ubiquity and complexity of graph-structured data in various real-world scenarios. However, current mainstream anomaly detection methods are primarily designed for centralized settings, which may pose privacy leakage risks in certain sensitive situations. Although federated graph learning offers a promising solution by enabling collaborative model training in distributed systems while preserving data privacy, a practical challenge arises as each client typically possesses a limited amount of graph data. Consequently, naively applying federated graph learning directly to anomaly detection tasks in distributed environments may lead to suboptimal performance results. We propose a federated graph anomaly detection framework via contrastive self-supervised learning (CSSL) [federated CSSL anomaly detection framework (FedCAD)] to address these challenges. FedCAD updates anomaly node information between clients via federated learning (FL) interactions. First, FedCAD uses pseudo-label discovery to determine the anomaly node of the client preliminarily. Second, FedCAD employs a local anomaly neighbor embedding aggregation strategy. This strategy enables the current client to aggregate the neighbor embeddings of anomaly nodes from other clients, thereby amplifying the distinction between anomaly nodes and their neighbor nodes. Doing so effectively sharpens the contrast between positive and negative instance pairs within contrastive learning, thus enhancing the efficacy and precision of anomaly detection through such a learning paradigm. Finally, the efficiency of FedCAD is demonstrated by experimental results on four real graph datasets.

Index Terms—Anomaly detection, attributed networks, contrastive self-supervised learning (CSSL), federated learning (FL).

Manuscript received 14 September 2022; revised 2 April 2023 and 9 April 2024; accepted 10 June 2024. Date of publication 20 June 2024; date of current version 5 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62072409, in part by the Zhejiang Provincial Natural Science Foundation under Grant LR21F020003, and in part by the Zhejiang Lab under Grant 2022NF0AC01. The work of Sajal K. Das was supported by the U.S. National Science Foundation (NSF) under Award CNS-2008878, Award SaTC-2030624, Award OAC-2104078, and Award EPCN-2319995. (Corresponding author: Xiaoran Yan.)

Xiangjie Kong and Wenyi Zhang are with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou 310023, China (e-mail: xjkong@ieee.org; wyzhang98@outlook.com).

Hui Wang, Mingliang Hou, and Xin Chen are with the School of Software, Dalian University of Technology, Dalian 116620, China (e-mail: wanghui.sd@outlook.com; teemohold@outlook.com; xin.chen.jx@outlook.com).

Xiaoran Yan is with the Research Institute of Artificial Intelligence, Zhejiang Lab, Hangzhou 311100, China (e-mail: yanxr@zhejianglab.com).

Sajal K. Das is with the Department of Computer Science, Missouri University of Science and Technology, Rolla, MO 65409 USA (e-mail: sdas@mst.edu).

Digital Object Identifier 10.1109/TNNLS.2024.3414326

NOMENCLATURE

$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$	Attribute network.
$\mathcal{V}$	Node set.
$\mathcal{E}$	Edge set.
$\mathbf{X} \in \mathbb{R}^{n \times f}$	Attribute matrix.
$\mathbf{A} \in \mathbb{R}^{n \times n}$	Adjacency matrix.
$\mathbf{x}_i \in \mathbb{R}^f$	Attribute vector of the i th node.
$P_i = (v_i, \mathcal{G}_i, y_i)$	Contrastive instance pair.
v_i	Target node of P_i .
$\mathcal{G}_i$	Local subgraph of P_i .
$y_i \in \{0, 1\}$	Label of P_i .
$\mathbf{A}_i \in \mathbb{R}^{c \times c}$	Adjacency matrix of $\mathcal{G}_i$.
T_c	Number of training epochs of FedAD.
$\mathbf{E}_i \in \mathbb{R}^{c \times d}$	Embedding matrix of $\mathcal{G}_i$.
$\mathbf{e}_i^{\text{lg}} \in \mathbb{R}^d$	Embedding vector of $\mathcal{G}_i$.
$\mathbf{e}_i^{\text{in}} \in \mathbb{R}^d$	Embedding vector of v_i .
s_i	Predicted score of P_i .
$\mathbf{W}^{(\ell)} \in \mathbb{R}^{(\ell-1) \times d^{(\ell)}}$	Learnable parameter of the ℓ th layer of GCN module.
$\mathbf{W}^{(d)} \in \mathbb{R}^{d \times d}$	Learnable parameter of discriminator module.
M	Number of clients.
T_{fc}	Rounds of communication between the client and the server.
T_{fed}	Number of training epochs of local discriminator module in federated contrastive self-supervised learning anomaly detection framework (FedCAD).
$\mathcal{G}_{C_i} = (\mathcal{V}_{C_i}, \mathcal{E}_{C_i}, \mathbf{X}_{C_i})$	Subgraph owned by client C_i .
$\tilde{Y}$	Pseudo labels.
$\mathbf{U}_{C_i}$	Neighbor embedding matrix of client C_i .

I. INTRODUCTION

ANOMALY, a fundamental research problem in machine learning, has been extensively explored across various domains, such as image and time series analysis [1]. Its primary goal is to identify data points or entities that exhibit unexpected behavior or do not conform to typical patterns. Traditional anomaly detection methods have mainly focused on single-dimensional data or simple relationships. However, the importance of attribute graph anomaly detection has become

increasingly evident with the advent of complex relational data in areas such as social networks, IoT, and biological networks.

Attribute graphs represent complex data structures that include nodes, edges, and various attribute information attached to each node and edge. In contrast to conventional graphs that only represent interactions between nodes, attribute graphs incorporate rich feature information for each node, allowing for the modeling of more intricate interaction systems [2]. Anomaly detection on attribute graphs involves identifying abnormal nodes, edges, or substructures within this more complex context. Unlike single-dimensional or tabular data, anomaly detection on attribute graphs poses more significant challenges. The detection of abnormal nodes in attribute graphs holds significant implications for various security-related applications and has become an urgent research topic in recent years. Examples include social network analysis, financial fraud detection, and cybersecurity. With the advancements in graph neural networks (GNNs), attribute graph anomaly detection has made significant progress and has demonstrated promising performance in many real-world complex scenarios [2], [3].

Despite the progress made by conventional centralized anomaly detection methods, they often require direct access to potentially sensitive user data, raising serious privacy concerns [3]. The following is an example of a common scenario. Considering the practical problems of the financial industry, for a variety of reasons, urban residents visit different banks. Therefore, their customer information, transaction network, and default history are only stored in the banks they visit. A robust anomaly detection model to perform efficient inference across the customer network, which contains all subgraphs from different banks, is a key requirement for banks wishing to collaborate to conduct comprehensive credit assessments of their customers and identify a common industry blacklist. Concerns about user privacy and conflicts of interest, it is difficult for all banks to share the customer network for training anomaly detection models. Two challenges arise in the above situation. The first challenge is data privacy protection. How do we train anomaly detection models in a distributed system while protecting data privacy? The second challenge is the small client subgraph. In a distributed system, the data scale of each client is small, and the data between clients may overlap. How do we increase the data scale of each client to improve the effect of anomaly detection?

Federated graph learning emerges as a compelling alternative to this issue by facilitating collaborative model training across distributed systems without compromising the privacy of individual data owners [4]. It does so by allowing clients to maintain their data locally and only share model parameters or encrypted gradients. However, when directly applied to anomaly detection tasks in distributed environments where each client holds a relatively small portion of the overall graph data, federated graph learning may suffer from performance degradation due to the lack of comprehensive data exposure during the learning process [5].

In this article, we propose FedCAD, a federated graph anomaly detection framework that integrates the ideas of federated graph learning and anomaly detection. For the first challenge of data privacy protection, a standard solu-

tion is to support computation over encrypted data using cryptography-based techniques, such as homomorphic encryption. While providing a solid security guarantee, these techniques have a substantial computational overhead, making them an inappropriate choice. FedCAD solves this challenge by designing federated parameter aggregation and anomaly information updates between clients based on federated learning (FL) without heavy cryptographic operations. First, FedCAD indirectly obtains the graph structure of other clients through federated parameter aggregation and integrates it into its client graph structure. Second, FedCAD does not directly share node features but obtains the neighbor subgraph of the target node through contrastive learning sampling and embeds the subgraph into the low-dimensional embedding vector before sharing. Meanwhile, the server only uploads the abnormal node neighborhood subgraph embeddings, but not the normal node neighborhood subgraph embeddings. These two points ensure data privacy security.

For the second challenge, that the client-side subgraphs are small in size, an evident approach is to scale-up the data per client, with the company or organization augmenting the scale of the dataset through other channels. However, expanding the scale of the dataset is costly. Another method is to perform data enhancement. However, most current data enhancement methods are suitable for computer vision (CV) and natural language processing (NLP) fields. Because graphs are complicated and non-Euclidean, few data enhancement techniques are available for graph data. In addition, the data generated by data augmentation are mostly negative examples, degrading the model performance in anomaly detection tasks. FedCAD solves this challenge by using a local anomaly neighbor embedding aggregate mechanism, which enables the current client to aggregate the anomaly node neighbor embeddings of other clients, expands the difference between the anomaly node and its neighbor nodes, and makes the difference between positive instances pairs and negative instances pairs in contrastive learning obvious, to improve the effectiveness of subsequent contrastive learning anomaly detection.

In this article, we propose FedCAD. The following is a summary of our major contributions.

- 1) We propose FedCAD, a federated graph anomaly detection framework. FedCAD can collaborate on distributed graph data to train high-quality graph anomaly detection models while protecting data privacy.
- 2) We propose an anomaly information update strategy that enables each client to aggregate anomaly node neighbor embeddings from other clients during federated processing. By doing so, it expands the difference between the anomaly node and its neighbor nodes, and makes the difference between positive instances pairs and negative instances pairs in contrastive learning obvious, to improve the effectiveness of subsequent contrastive learning anomaly detection.
- 3) We evaluate the proposed FedCAD on four datasets. The results verify the performance of the algorithm and its superiority over baseline methods.

The structure of the remainder of this article is as follows. We analyze the related work in Section II. The preliminary definitions, notations, and problem statements are explained

in Section III. We illustrate the technical details of FedCAD in Section IV. We show and analyze the experimental results in Section V. We make a conclusion of this article in Section VI.

II. RELATED WORK

In this section, we outline the most related work: anomaly detection on attributed networks, contrastive self-supervised learning (CSSL), and federated graph learning.

A. Anomaly Detection on Attributed Networks

Due to the wide application of attribute networks in complex system modeling, attribute network anomaly detection has gradually emerged in recent years. Some shallow methods are used for attribute network anomaly detection. Perozzi and Akoglu [6] detect anomalies on the attribute network by extracting the self-network information of each node. Peng et al. [7] add CUR decomposition to the residual analysis to perform anomaly detection on the attribute network to alleviate the adverse effect of noise features on anomaly detection. Unfortunately, because of the limitations of shallow mechanisms, these models cannot capture complex interactions between different information patterns, especially when the dimensionality of the features is high [8].

GNNs are a class of deep neural networks [9] specifically designed to model relational patterns inherent in non-Euclidean graph-structured data. Subsequently, a succession of spectral-based GNN variants emerged, harnessing filter from graph signal processing viewpoint [10]. Among these, graph convolutional network (GCN) stands out by implementing a localized first-order approximation of the spectral graph convolution operations [11]. This strategic approach enables the efficient extraction of meaningful node representations. Due to the rapid development of deep learning-based anomaly detection, researchers have also proposed anomaly detection methods on attribute networks. Ding et al. [8] calculated anomalies based on the reconstruction error of the adjacency matrix and attribute matrix by an autoencoder with a GCN layer. Fan et al. [12] propose a framework that learns the cross-modal interaction between the network structure and node attributes. Li et al. [13] employ a spectrogram autoencoder to extract each node's latent embedding and a Gaussian mixture model (GMM) to detect each node. Yu et al. [14] employ random walk sampling and an autoencoder model to dynamically learn network embedding and use a cluster-based detector to detect anomalies. Liu et al. [15] overcome the shortcomings of autoencoders that cannot fully utilize data and cannot operate on large-scale network data. It uses a CSSL-based anomaly detection framework to capture the connection between the target node and its adjacent substructures. The anomaly node is detected statistically by measuring the consistency of each instance pair with its output score through contrastive learning.

B. Contrastive Self-Supervised Learning

CSSL is a pivotal branch of self-supervised learning in which models are trained to discern similarities and dissimilarities among nodes without explicit labels [16]. The remarkable achievements of CSSL in CV have spurred its progressive adoption within graph-based learning paradigms.

DGI constructs contrastive instance pairs by pairing a node's representation with a graph-level summary vector, exploiting graph corruption to generate negative samples and thereby enabling the node representation learning in an unsupervised setting [17]. Graph contrastive coding (GCC) initializes GNNs by pretraining on universal graph datasets, using a strategy that involves sampling two subgraphs per node as a positive instance pair and utilizing the information noise contrastive estimation (InfoNCE) loss function for representation learning [18]. Graphical mutual information (GMI) focuses on optimizing the agreement between a node's embeddings and the raw attributes of its neighbors and between the embeddings of connected nodes [19]. However, most CSSL approaches do not directly target anomaly detection despite their efficacy in learning data representations. To adapt to anomaly detection, CSSL framework for anomaly detection on attributed network (CoLA) adapts the entire contrastive learning framework to calculate anomaly scores for individual nodes [15]. Recognizing that conventional instance pair definitions might inadequately capture node anomalies, CoLA proposes a new form of contrastive instance pair tailored for graph contrastive learning. This novel design emphasizes local information of each node over global properties. While extensive research has demonstrated the broad applicability of graph contrastive learning, the challenge of applying it to anomaly detection in attributed networks in distributed environments remains a novel and complex research problem.

C. Federated Graph Learning

Federated learning is a novel distributed learning paradigm that can jointly model with multiple clients [20], [21]. Federated learning has been extensively studied in many fields, for instance, image recognition [22], vehicle localization [23], machinery fault diagnosis [24], wireless communication [25], and edge computing [26]. Federated graph learning is a direction of federated learning and has received extensive attention. Federated graph learning is an emerging technology that can jointly train machine learning models on graph datasets distributed to multiple clients to ensure user privacy [27]. Caldarola et al. [28] use GCN to model interdomain interactions and study cross-domain heterogeneity in FL. He et al. [29] propose a general federation framework suitable for graph embedding algorithms. Jiang et al. [30], Chen et al. [31], and Wu et al. [32] study the privacy issue of federated GNNs. Wang et al. [33] introduce model-agnostic meta-learning (MAML) into graph FL to maintain model generality while dealing with non-independent and identically distributed (IID) graph data. Zhang et al. [34] study the missing neighbor synthesis issue in the subgraph FL context. Xie et al. [35] study graph classification to reduce heterogeneity in the structure and features of graphs owned by local systems. Peng et al. [36] focus on research to strengthen the privacy protection of federated learning and study the use of differential privacy for federated knowledge graph embedding.

III. PRELIMINARIES

A. Notations and Definitions

The notations are summarized in Nomenclature. For notation, we use bold lowercase representing a vector (e.g., $\mathbf{x}$),

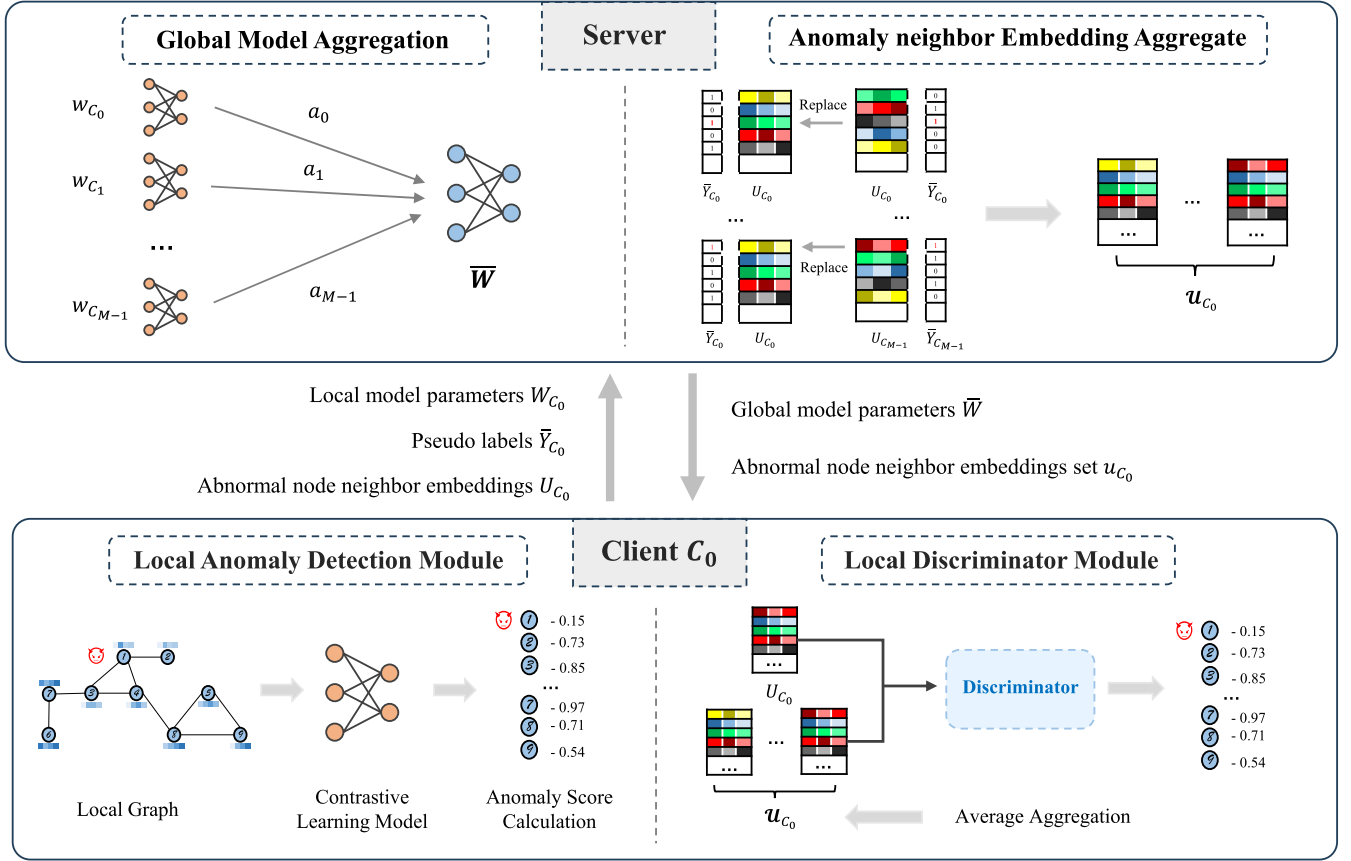


Fig. 1. Overall framework of FedCAD. First, the node anomaly score was calculated by the local anomaly detection model, and the client uploaded the local anomaly detection model parameters W to the server. Second, the server performs global model parameter aggregation and sends $\bar{W}$ to each client to complete the local anomaly detection model parameter update and repeat T_{ic} times. Finally, the pseudo label $\tilde{Y}$ was discovered according to the confidence threshold λ . Each client uploads the pseudo label $\tilde{Y}$ and the neighbor embedding matrix of the abnormal node U to the server. The server updates the neighbor embedding of the abnormal node and sends the updated neighbor embedding matrix set $\mathcal{U}$ of the abnormal node to each client. The client obtains the final node anomaly score through the local discriminator model.

bold uppercase representing a matrix (e.g., $\mathbf{X}$), and calligraphy letter denoting a set (e.g., $\mathcal{V}$).

Definition 1 (Attributed Networks): $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ is denoted as an attributed network, where $\mathcal{V}$ is the node set, $\mathcal{E}$ is the edge set, and $\mathbf{X} \in \mathbb{R}^{n \times f}$ is the attribute matrix. The i th row vector $\mathbf{x}_i \in \mathbb{R}^f$ of the attribute matrix represents the attribute vector of the i th node. The structure of the attribute network is represented as a binary adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, where the existence of a link between nodes v_i and v_j is represented by $\mathbf{A}_{i,j} = 1$, otherwise $\mathbf{A}_{i,j} = 0$. An attributed network can also be denoted as $\mathcal{G} = (\mathbf{A}, \mathbf{X})$.

Definition 2 (Federated Learning Client Subgraph): We have a server and M clients in the FL system. $\mathcal{G}_{C_i} = (\mathcal{V}_{C_i}, \mathcal{E}_{C_i}, \mathbf{X}_{C_i})$ is a subgraph of client C_i , for $C_i \in [M]$. Note that there are certain differences in the number of nodes and graph structure of each client, but there are also some overlapping nodes between the clients. That is, for any graph $\mathcal{G}_{C_k}$, there exists $k \neq t$, so that $\mathcal{V}_{C_k} \cap \mathcal{V}_{C_t} \neq \emptyset$. This setup conforms to the distribution of graph data in the real world.

B. Problem Statement

1) **Node Anomaly Detection in Distributed Systems:** Suppose there are M clients, $\mathcal{G}_{C_k}$ denotes the data owned by the

client C_k , the goal is to train an anomaly score function f in distributed systems. The anomaly nodes can be detected by ranking each node according to its anomaly score. We employ an unsupervised anomaly detection method. In the training phase, we only give the attribute network $\mathcal{G}$ containing abnormal nodes and do not give the abnormal labels of the nodes.

In this work, we aim to propose a federated anomaly detection framework on attributed networks, which can collaboratively learn on isolated subgraphs in different clients to train a high-quality anomaly detection model while protecting data privacy.

IV. DESIGN OF FEDCAD

A. Overall Architecture

We propose FedCAD, a novel federated CSSL anomaly detection framework on attributed networks, as shown in Fig. 1.

First, we adopt an anomaly detection framework on attributed networks, which can model the relationship between the target node and its partial neighboring substructure by training a contrastive learning model, revealing various anomalies in the graph. Second, we integrate the anomaly detection algorithm into the federated learning framework to implement

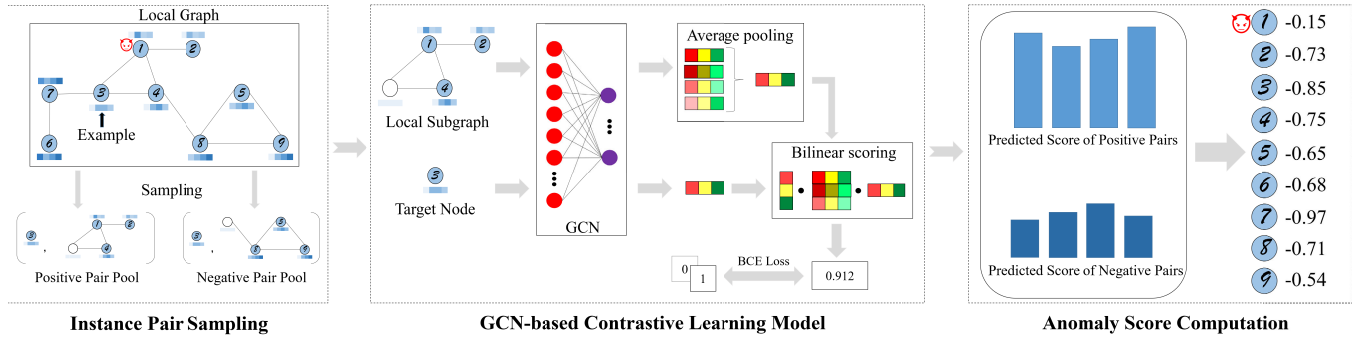


Fig. 2. Framework of the local anomaly detection. Here we assume that node v_1 is an abnormal node and the others are normal nodes. Node v_3 is the anomaly evaluation target node. First, taking v_3 as the target node, positive instance pairs and negative instance pairs are obtained by multiple sampling. The GCN-based contrastive learning model then computes the prediction score for each instance pair. Finally, the anomaly score of node v_3 was calculated through the anomaly score calculation module.

a common federated graph anomaly detection framework, called FedAD. It simply combines the local anomaly detection model with the classic federated learning algorithm Fedavg [37], allows the local client to upload the parameters of the anomaly detection model, performs the average aggregation of the parameters in the server to update the parameters, and finally sends the updated parameters to each client. Third, based on FedAD, we propose FedCAD. FedCAD employs federated learning to upload the vector embedding of the neighbor subgraph of the target node sampled by contrastive learning in the client to the server. The server aggregates abnormal information between clients and sends the aggregated vector embedding set to each client. The client performs average aggregation, thereby improving the difference in the relationship between the target node and the neighbor subgraph in contrastive learning and improving the effectiveness of local anomaly detection.

B. Local Anomaly Detection Module

Inspired by CoLA [15], we adopt an anomaly detection module, which is shown in Fig. 2. The overall procedure is depicted in Algorithm 1.

- 1) *Instance Pair Sampling*: We adopt a contrastive instance pair. The first element is a single node called the “target node.” The target node randomly traverses every node on the graph in an epoch. The second element is a subgraph sampled from the “target node.” Node anomalies in attribute networks are usually reflected as inconsistencies between the target node and its local neighbors.
- 2) *GCN-Based Contrastive Learning Model*: The instance pair P_i can be denoted as follows:

$$P_i = (v_i, \mathcal{G}_i, y_i) \quad (1)$$

where v_i is the target node, and $\mathcal{G}_i$ is the subgraph sampled from v_i . $\mathcal{G}_i$ can be denoted as

$$\mathcal{G}_i = (\mathbf{A}_i, \mathbf{X}_i) \quad (2)$$

and y_i is the label of P_i . y_i can be denoted as follows:

$$y_i = \begin{cases} 1, & P_i \text{ is a positive instance pair} \\ 0, & P_i \text{ is a negative instance pair.} \end{cases} \quad (3)$$

Algorithm 1 Overall Procedure of the Local Anomaly Detection Module

Input: The attributed network: $\mathcal{G}$, The training epochs: T_c , The batch size: B , The sampling rounds: R .

Output: Anomaly score function: $f(\cdot)$.

- 1: Initialize the contrastive learning model parameters $(\mathbf{W}^{(0)}, \dots, \mathbf{W}^{(d)})$.
- 2: // The train stage.
- 3: **for** $t = 1, 2, \dots, T_c$ **do**
- 4: Initialize the batches $\mathcal{B}$.
- 5: Sample positive instance pairs $(P_1^{(+)}, \dots, P_B^{(+)})$.
- 6: Sample negative instance pairs $(P_1^{(-)}, \dots, P_B^{(-)})$.
- 7: Calculate the scores $(s_1^{(+)}, \dots, s_B^{(+)}, s_1^{(-)}, \dots, s_B^{(-)})$ by Eq. 7.
- 8: Calculate $\mathcal{L}_{cml}$ by Eq. 8.
- 9: Update the parameters $(\mathbf{W}^{(0)}, \dots, \mathbf{W}^{(d)})$ of contrastive learning model via back propagation.
- 10: **end for**
- 11: // The inference stage.
- 12: **for** $v_i \in \mathcal{V}$ **do**
- 13: Sample positive instance pairs $(P_{i,1}^{(+)}, \dots, P_{i,R}^{(+)})$.
- 14: Sample negative instance pairs $(P_{i,1}^{(-)}, \dots, P_{i,R}^{(-)})$.
- 15: Calculate the scores $(s_{i,1}^{(+)}, \dots, s_{i,R}^{(+)}, s_{i,1}^{(-)}, \dots, s_{i,R}^{(-)})$ by Eq. 7.
- 16: Calculate the anomaly score $f(v_i)$ by Eq. 10.
- 17: **end for**

In positive instance pair, the target node and the subgraph are connected. In negative instance pair, the target node and the subgraph are not connected.

First, the local subgraph is fed into GCN to obtain the local subgraph embedding. The GCN model can be written as follows:

$$\mathbf{E}_i = \phi\left(\tilde{\mathbf{D}}_i^{-\frac{1}{2}} \tilde{\mathbf{A}}_i \tilde{\mathbf{D}}_i^{-\frac{1}{2}} \mathbf{X}_i \mathbf{W}^{(0)}\right) \quad (4)$$

where $\tilde{\mathbf{D}}_i$ is the degree matrix of the local subgraph, $\mathbf{X}_i$ is the attribute matrix, and $\mathbf{E}_i$ is the embeddings of subgraph nodes.

Then, the embeddings of subgraph nodes $\mathbf{E}_i$ are transformed into a local subgraph embedding vector $\mathbf{e}_i^{\text{lg}}$ by

the average pooling method. Specifically, the method is written as follows:

$$\mathbf{e}_i^{\text{lg}} = \sum_{k=0}^{n_i-1} \frac{(\mathbf{E}_i)_k}{n_i} \quad (5)$$

where n_i is the number of nodes in the subgraph $\mathcal{G}_i$. We also map the target node to the same embedding space to compare them

$$\mathbf{e}_i^{\text{tn}} = \phi(\mathbf{x}_{v_i} \mathbf{W}^{(0)}) \quad (6)$$

where $\mathbf{W}^{(0)}$ is the mapping weight matrix of GCN. $\mathbf{x}_{v_i}$ is the attribute vector of target node, and $\mathbf{e}_i^{\text{tn}}$ is the target node embedding.

We evaluate the final prediction score by comparing $\mathbf{e}_i^{\text{lg}}$ and $\mathbf{e}_i^{\text{tn}}$

$$s_i = \sigma(\mathbf{e}_i^{\text{lg}} \mathbf{W}^{(d)} \mathbf{e}_i^{\text{tn}\top}) \quad (7)$$

where $\mathbf{W}^{(d)}$ is the discriminator weight matrix, and $\sigma(\cdot)$ is the logistic sigmoid function.

As the objective function, we use standard binary cross entropy (BCE) loss. Concretely, the objective function is denoted as follows:

$$\mathcal{L}_{\text{cml}} = - \sum_{i=1}^B y_i \log(s_i) + (1 - y_i) \log(1 - s_i). \quad (8)$$

- 3) *Anomaly Score Computation*: Typically, well-trained GNN models tend to capture the matching patterns of normal samples more effectively, as they make up a large proportion of the training data and have distinct regularities and features. Conversely, abnormal samples present a greater challenge due to their variety and irregularity, making it harder for the model to fit their patterns. Ideally, for a normal node, the predicted score of its positive pair $s_i^{(+)}$ should approximate 1, while the negative one $s_i^{(-)}$ approaches 0. For an abnormal node, the discrimination between the predicted scores of its positive and negative pairs would be lower since the model is not able to distinguish its matching pattern well. With the properties mentioned above, we can define the anomaly score for each node as simply the difference between its negative and positive scores. This score is negative for normal nodes, whereas it is close to 0 for abnormal nodes. Hence, the higher the anomaly score, the greater the probability of the node being anomalous. The anomaly score for each node v_i is written as follows:

$$f(v_i) = s_i^{(-)} - s_i^{(+)} \quad (9)$$

where $s_i^{(+)}$ is the predicted score of the positive pair, and $s_i^{(-)}$ is the predicted score of the negative pair. However, a single-sampled local subgraph of the target node cannot describe the entire neighborhood distribution, which causes the performance to decrease. To solve this problem, we repeatedly run multiple rounds to calculate the average prediction scores. The final anomaly score for each node is written as follows:

$$f(v_i) = \frac{\sum_{r=1}^R (s_{i,r}^{(-)} - s_{i,r}^{(+)})}{R} \quad (10)$$

where R is the sampling round.

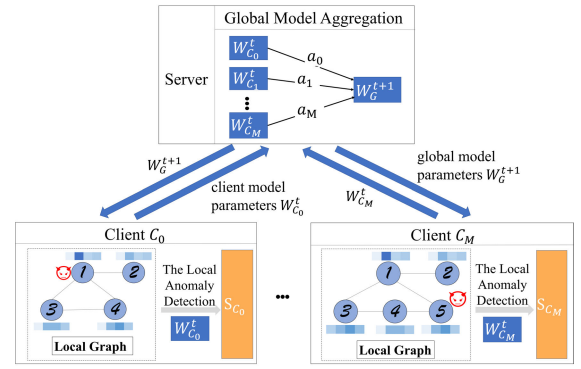


Fig. 3. Overall framework of FedAD. In the communication between the client and server in round t , each client uploads local anomaly detection model parameters $W_{C_k}^t$ to the server, and the server performs global model parameter aggregation and sends W_G^{t+1} to each client to complete the local anomaly detection model parameter update.

Algorithm 2 Algorithm of FedAD

Input: Attributed network: $\{\mathcal{G}_{C_k} = (\mathcal{V}_{C_k}, \mathcal{E}_{C_k}, \mathbf{X}_{C_k})\}_{C_k=0}^{M-1}$, The rounds of communication between the client and the server: T_{fc} .

Output: Anomaly score function: $f(\cdot)$.

- 1: The server initializes global model parameters W_G^0 , distributing them to each client.
- 2: **while** not converge and $t_{fc} \leq T_{fc}$ **do**
- 3: // Client: local model training.
- 4: Run the train stage of Algorithm 1.
- 5: Upload $W_{C_k}^{t_{fc}}$ to the server.
- 6: // Server: global model aggregation.
- 7: Update global model $W_G^{t_{fc}+1}$ by Eq. 12.
- 8: Distribute $W_G^{t_{fc}+1}$ to each client.
- 9: $t_{fc} \leftarrow t_{fc} + 1$.
- 10: **end while**

C. FedAD Algorithm

McMahan et al. [37] proposed FedAvg. The FedAvg is the first basic FL algorithm. We integrate the anomaly detection algorithm into the federated learning framework to implement a simple federated graph anomaly detection framework, called FedAD, as shown in Fig. 3. Specifically, suppose there are M clients, $\mathcal{G}_{C_k}$ denotes the data owned by the client C_k , and W_{C_k} denotes the local anomaly detection module trained on the client C_k . W_G denotes the global model on the server. A total of T_c rounds of communication between the client and the server. The overall procedure is depicted in Algorithm 2. The FedAD algorithm process is the following steps.

- 1) *Initialization*: The server initializes hyperparameters and the local anomaly detection module parameters $W_{C_k}^0$, and so on, and distributes them to each client.
- 2) *Local Anomaly Detection Module Training*: After receiving the local anomaly detection parameters from the server, each client trains the local anomaly detection model and updates the local anomaly detection parameters using local data. For the client C_k , the t th communication is

$$W_{C_k}^t = W_{C_k}^t - \alpha \nabla L_{\mathcal{G}_{C_k}}(W_{C_k}^t) \quad (11)$$

where α is the learning rate, and the updated local model parameters $W_{C_k}^t$ are uploaded to the server.

- 3) *Global Model Parameters Update*: The server aggregates the parameters of the updated local model uploaded by each client to generate the updated global model parameters W_G^{t+1} . The formula is

$$W_G^{t+1} = \sum_{C_k=0}^{M-1} \frac{|\mathcal{G}_{C_k}|}{\sum_{C_k=0}^{M-1} |\mathcal{G}_{C_k}|} W_{C_k}^t \quad (12)$$

where $|\mathcal{G}_{C_k}|$ represents the number of samples of the client C_k . The updated global model parameters W_G^{t+1} are distributed to each client to continue the next round of training.

The whole process will repeat steps 2) and 3) until the global model converges or reaches the maximum number of communications.

FedAD combines the local anomaly detection model with the classic federated learning algorithm FedAvg. First, FedAD is an integral part of our proposed federated anomaly detection FedCAD, providing FedCAD with the anomaly node score, the target node embedding vector, and the neighbor node embedding matrix. Second, FedAD is an ablation version of FedCAD, which is used to verify the effectiveness of pseudo-label discovery, local anomaly neighbor embedding aggregate, and local discriminator module.

D. Anomaly Information Update Between Clients Based on FL

In graphs, connections may generally exist between nodes and their neighbors. Conversely, an anomaly usually occurs when there is a mismatch between the target node and its neighbors, violating the network's original matching pattern.

Contrastive learning, with its intrinsic discriminative mechanism, is adept at capturing abnormal mismatch patterns. However, the limited scale of data within each client presents practical challenges for our study. In the context of banking operations, for example, bank branches are typically distributed throughout the country, with some branches located in sparsely populated areas possessing relatively small user data. This scarcity of graph data leads to an overabundance of duplicate nodes when forming positive and negative instance pairs. Consequently, when contrastive learning samples negative instance pairs from such limited data, the neighboring nodes in these pairs are highly likely to overlap with those in the positive instance pairs. As a result, this leads to inconspicuous distinctions between positive and negative instance pairs, ultimately diminishing the effectiveness of contrastive learning. Despite our attempts to mitigate this issue through repeated execution of R rounds of contrast instance pair sampling in the local anomaly detection module, this approach fails to address the problem comprehensively.

To solve the problem, we propose FedCAD. FedCAD extra aggregates the neighbor node embeddings of anomalous nodes from other clients during the federated learning process. This strategy enlarges the difference between abnormal nodes and their neighboring counterparts, rendering the contrast between positive and negative instance pairs more apparent,

Algorithm 3 Algorithm of FedCAD

Input: Attributed network: $\{\mathcal{G}_{C_k} = (\mathcal{V}_{C_k}, \mathcal{E}_{C_k}, \mathbf{X}_{C_k})\}_{C_k=0}^{M-1}$, The rounds of communication between the client and the server: T_{fc} , The number of training epochs of Local Discriminator Module: T_{fcd} , The confidence threshold for determining the pseudo label: λ .

Output: Anomaly score function: $f(\cdot)$.

Server initialize global model parameters W_G^0 , distributing them to each client.

Run the Algorithm 2.

while not converge and $t_{fc} \in 1, 2, \dots, T_{fc}$ **do**

// Client: Calculate the pseudo labels and extract the neighbor embedding matrix.

Run the inference stage of Algorithm 1.

Calculate the pseudo labels $\bar{Y}_{C_k}$ by Eq. 13.

Extract the neighbor embedding matrix $\mathbf{U}_{C_k}$.

Upload $\bar{Y}_{C_k}$ and $\mathbf{U}_{C_k}$ to the server.

// Server: local anomaly neighbor embedding aggregate.

Aggregate the local anomaly neighbor embedding matrix by Eq. 14.

Distribute $\mathcal{U}_{C_k} = \{\mathbf{U}_{C_k}^{C_0}, \mathbf{U}_{C_k}^{C_1}, \dots, \mathbf{U}_{C_k}^{C_k}, \dots, \mathbf{U}_{C_k}^{C_{M-1}}\}$ to client C_k .

// Client: local discriminator module training.

// Training phase.

Average the aggregated neighbor embedding matrices by Eq. 15.

for not converge and $t_{fcd} \in 1, 2, \dots, T_{fcd}$ **do**

Calculate the predicted scores s_i of v_i by Eq. 16.

Calculate $\mathcal{L}_{fin}$ by Eq. 17.

Update the local discriminator module parameters

$W_{C_k}^{t_{fc}}$ via back propagation.

end for

// Inference phase.

Calculate the predicted scores s_i of v_i by Eq. 16.

Calculate the anomaly score $f(v_i)$ by Eq. 9.

end while

thus improving the effectiveness of contrastive anomaly detection. We first design a pseudo-label discovery on clients and local anomaly node embedding update on the server. Then, we feed the obtained results into the discriminator to get the final abnormal node score S . The algorithm of the FedCAD framework is depicted in Algorithm 3.

E. Pseudo-Label Discovery

Based on the anomaly node score of FedAD, we try to discover pseudo labels of the anomaly node. To get the pseudo labels, we specifically extract these prediction scores from the final anomalous node score S . For the prediction results from S_k of the i th node in S , if the predicted score is higher than the threshold, then it is a pseudo label

$$\bar{Y}_{C_i}^k = \begin{cases} 1, & \text{if } (S_k > \lambda) \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

where $\bar{Y}_{C_i}^k$ is the pseudo label of the k th target node of the client C_i , and $\lambda \in (-1, 0)$ is the confidence threshold for

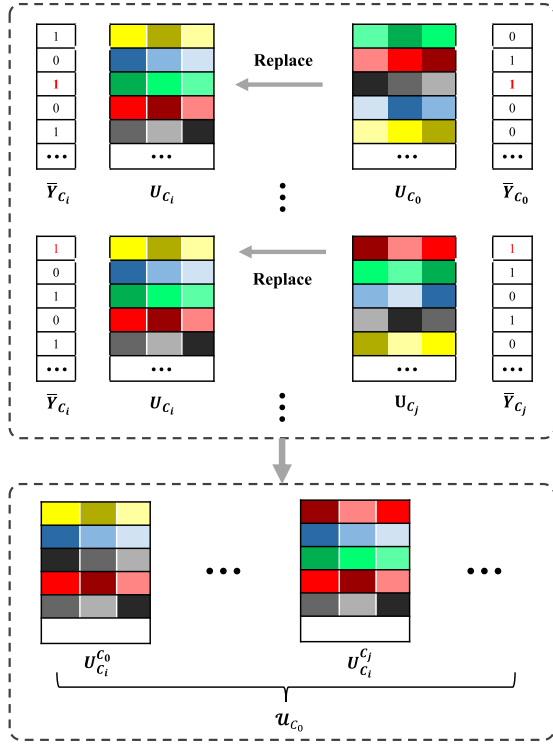


Fig. 4. Illustrative process of local anomaly neighbor embedding aggregate. $\bar{Y}_{C_i}$ and $\bar{Y}_{C_j}$ are the pseudo labels of client C_i and client C_j , and U_{C_i} and U_{C_j} are the abnormal node neighbor embedding matrices of client C_i and client C_j . In the local abnormal neighbor embedding aggregation, when $\bar{Y}_{C_i}^k = \bar{Y}_{C_j}^k = 1$, let the neighbor embedding vector $U_{C_0}^k = U_{C_j}^k$ of the corresponding abnormal node to obtain $U_{C_i}^{C_j}$.

determining the pseudo label. Then, we extract the neighbor node embedding matrix U_{C_i} of all target nodes, which consists of the local subgraph embedding vector $\mathbf{e}_i^{\text{lg}}$. $\mathbf{e}_i^{\text{lg}}$ is converted from the readout module in FedAD, which is shown in (5). We set the vector in $U_{C_i}^k$ with $\bar{Y}_{C_i}^k = 0$ as the 0 vectors. There are two main reasons. First, it can effectively protect data privacy. Only uploading the abnormal node neighbor embedding vector can effectively protect data privacy from being leaked. Second, it greatly reduces the communication overhead of federated learning. The proportion of abnormal nodes is small, and only uploading the neighbor embedding vector of abnormal nodes can save a lot of communication overhead. In the final step, each client uploads U and $\bar{Y}$ to the server.

1) *Local Anomaly Neighbor Embedding Aggregate*: After the server receives U and $\bar{Y}$ from each client update, the server obtains the pseudo label of the abnormal node that coincides with each client and other clients. After that, the server aggregates the abnormality neighbor embeddings of all clients according to the pseudo label of the coincident node. The process of local anomaly neighbor embedding aggregate is shown in Fig. 4. For the sake of illustration, consider an arbitrary client C_i among a total of M clients. Here, C_j , where $i \neq j$, represents any other distinct client from the set of all clients.

- 1) The pseudo label $\bar{Y}_{C_i}$ takes the intersection with the pseudo label $\bar{Y}_{C_j}$ of other clients to obtain the set

of coincident abnormal node pseudo labels $Y_{C_i} = \{\bar{Y}_{C_i} \cap \bar{Y}_{C_0}, \dots, \bar{Y}_{C_i} \cap \bar{Y}_{C_j}, \dots, \bar{Y}_{C_i} \cap \bar{Y}_{C_{M-1}}\} = \{\bar{Y}_{C_i}^{C_0}, \dots, \bar{Y}_{C_i}^{C_j}, \dots, \bar{Y}_{C_i}^{C_{M-1}}\}$.

- 2) We copy M copies of the neighbor embedding matrix U_{C_i} of the target node, and then replace the neighbor embedding matrix U_{C_i} with the neighbor embedding matrix U_{C_j} of other clients according to the obtained coincident abnormal node pseudo label set $\bar{Y}_{C_i}^{C_j}$, which is written as follows:

$$U_{C_i}^{C_j} = (\mathbf{I} - \bar{Y}_{C_i}^{C_j}) \odot U_{C_i} + \bar{Y}_{C_i}^{C_j} \odot U_{C_j} \quad (14)$$

where $\mathbf{I}$ is the identity matrix and $\odot$ is the Hadamard product, i.e., the multiplication of the corresponding positions of the two matrices. We then obtain the aggregated neighbor embedding matrix set $U_{C_i} = \{U_{C_i}^{C_0}, \dots, U_{C_i}^{C_j}, \dots, U_{C_i}^{C_{M-1}}\}$.

- 3) The server repeatedly executes the above process for each neighbor embedding uploaded by each client.

2) *Local Discriminator Module*: Fig. 5 shows the process of the local discriminator module. We send the aggregated neighbor embedding matrix set to their respective clients. Each client averages the set of aggregated neighbor embedding matrices, which enlarges the difference between the anomalous target node and its neighbors

$$U_{C_i} = \frac{1}{M} \sum_{C_j=0}^{M-1} U_{C_i}^{C_j} \quad (15)$$

where U_{C_i} is the averaged neighbor embedding matrix. $\mathbf{u}_i^{\text{lg}}$ denotes the averaged neighbor embedding vector of the i th node. Then, we obtain the target node embedding vector $\mathbf{e}_i^{\text{tn}}$, which is generated by (6). Next, we feed $\mathbf{u}_i^{\text{lg}}$ and $\mathbf{e}_i^{\text{tn}}$ into the local discriminator module for training. The local discriminator module also uses a sample bilinear scoring function, which is written as follows:

$$s_i = \text{Discriminator}(\mathbf{u}_i^{\text{lg}}, \mathbf{e}_i^{\text{tn}}) = \sigma(\mathbf{u}_i^{\text{lg}} \mathbf{W}^{(d)} \mathbf{e}_i^{\text{tn}\top}). \quad (16)$$

Here, we attempt to maximize similarities between the ground-truth label y_i and the prediction result s_i . Therefore, we adopt the standard BCE loss as our objective function

$$\mathcal{L}_{\text{fin}} = - \sum_{i=1}^B y_i \log(s_i) + (1 - y_i) \log(1 - s_i). \quad (17)$$

Finally, we compute the local anomaly score using the anomaly score calculation formula, as shown in (9).

F. Complexity Analysis

In FedCAD, we analyze the time complexity of this framework considering two main components: FedAD and the anomaly information update between clients based on FL. In FedAD, we consider two main components: the local anomaly detection module and FedAvg. For the local anomaly detection module, the time complexity of each RWR subgraph sampling is $O(c\delta)$, where c is the number of sampled neighbor nodes and δ is the mean degree of the network. We run R rounds of sampling for each node, then the total time

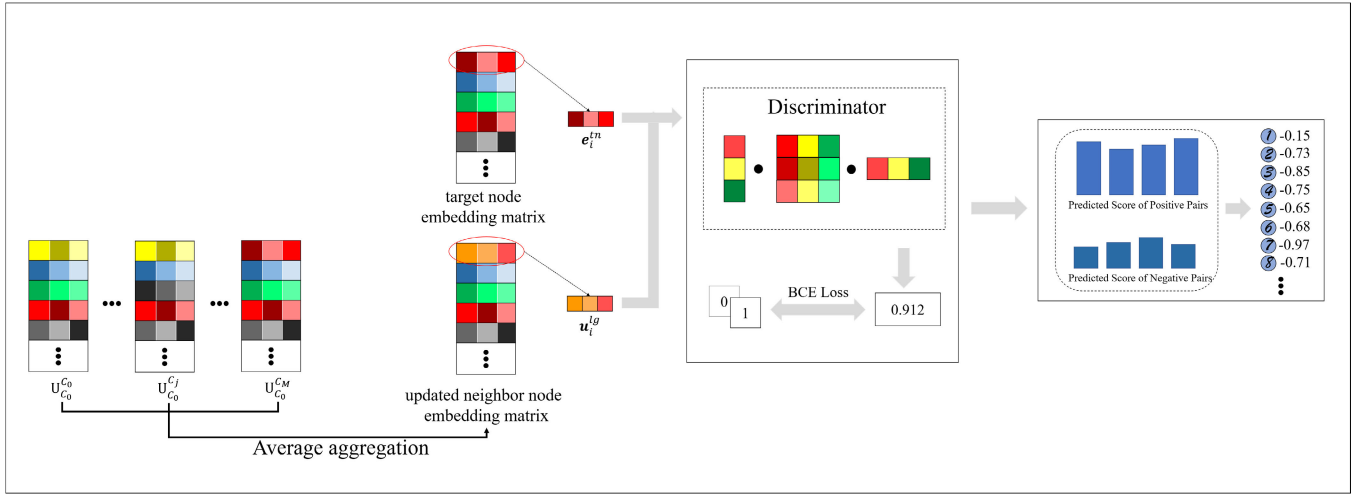


Fig. 5. Illustrative process of local discriminator module. The client aggregates the abnormal node neighbor embedding matrix set sent by the server on average, and then feeds the embedding vector $\mathbf{u}_i^{\text{lg}}$ of the neighbor nodes and the embedding vector $\mathbf{e}_i^{\text{m}}$ of the target node into the discriminator, and obtains the prediction score of the node through contrastive learning, and finally obtains the abnormal score of the node.

TABLE I
STATISTICS OF THE DATASETS

Dataset	# nodes	# edges	# attributes	# anomalies
Cora	2,708	5,429	1,433	134
Wiki	2,405	17,981	4,973	109
BlogCatalog	5,196	171,743	8,189	257
Flickr	7,575	239,738	12,407	377

complexity becomes $O(cn\delta R)$, where n is the number of graph nodes in the client. The time complexity of the GCN module is $O(c^2nR)$. The time complexity of the local anomaly detection module is $O(cnR(c + \delta))$. For FedAvg, it consists of two components: client and server. For the client, all clients run in parallel for each epoch, with the same time complexity as the local anomaly detection module. For the server, only the average aggregation operation of local parameters is performed, and the time complexity is $O(M)$. The rounds of communication between the client and the server are T_{fc} . Therefore, the overall time complexity of FedAD is $O(T_{\text{fc}}(cnR(c + \delta) + M))$. For the anomaly information updates between clients based on FL, the time complexity is mainly generated by the local anomaly node embedding update, which is $O(M^2)$. The overall time complexity of FedCAD is $O(T_{\text{fc}}(cnR(c + \delta) + M^2))$.

To reduce the communication complexity, when the client uploads the neighbor node embedding matrix, FedCAD only uploads the abnormal node neighbor embedding vector, and the normal node neighbor embedding vector is not uploaded. Since the proportion of abnormal nodes is small, it can save a lot of communication overhead.

V. EXPERIMENTS

A. Datasets

We employ four datasets that are commonly used to detect anomalies in attributed networks, with the basic information shown in Table I.

- 1) *Cora*: Cora is an academic citation network. The node represents the published paper, and the edge

represents the citation relationship between the papers. The attribute vector is represented by bag-of-words.

- 2) *Wiki*: Wiki is a web page link network derived from the English Wikipedia website. Each node is a web page explaining the term. The edge is as hyperlink reference between web pages. The content of web pages is transformed into bag-of-words representations of the initial node features.
- 3) *BlogCatalog*: BlogCatalog is a blog sharing website. Each node represents the users of websites. The edge represents the following relationships between users. Node attributes consist of a list of user tags.
- 4) *Flickr*: Flickr is an image hosting and sharing website. Each node represents the users of websites. The edge represents the following relationships between users. Node attributes consist of a list of user tags and photo tags.

The aforementioned datasets lack ground-truth anomalies, hence to evaluate them, artificial anomalies must be injected into the attribute network. We refer to the node attribute anomalies injection method that has been used in the previous research [15] to generate node attribute anomalies for each dataset. We randomly pick a node v_i as the target node. Then, we select another n nodes $V = (v_1, v_2, \dots, v_n)$ as a candidate set. Here, we set $n = 50$. For each $v_j \in V$, between the attribute vectors of x_i and x_j we compute their Euclidean distance. Then, we pick the node v_j with the largest Euclidean distance from the node v_i as the anomaly node. We change x_i to x_j .

We ultimately obtain the perturbed network using the above-mentioned injection technique, the number of abnormal nodes is shown in Table I. In our experiments, all class labels are removed and the anomaly labels y_i are only visible in the inference stage.

B. Experimental Settings

In this section, we outline the critical aspects of our experimental setup. This includes presenting the comparative

baselines, discussing the evaluation metrics used, and detailing the parameter setting in our framework.

1) *Baselines*: We compare our proposed FedCAD framework with the following commonly used methods for anomaly detection.

- 1) *DOMINANT* [8]: In DOMINANT, the adjacency matrix and the attribute matrix are concurrently reconstructed using an autoencoder. By calculating the reconstruction error, it evaluates the abnormality of each node.
- 2) *AnomalyDAE* [12]: AnomalyDAE is an anomaly detection framework via dual autoencoders, which learns the cross-modal interactions during node attribute reconstruction by taking attribute embeddings as inputs to attribute decoders. It detects anomalies by measuring the reconstruction error.
- 3) *CoLA* [15]: CoLA is an anomaly detection framework to catch the connections between the target node and its adjacent substructures based on CSSL in an unsupervised manner. The anomaly of each node is estimated statistically by measuring the consistency of each instance pair with its output score through contrastive learning.
- 4) *FedAD*: FedAD combines the localized model utilizing the weighted-average approach to obtain the global model. This method is an ablation version of FedCAD, which is used to verify the effectiveness of pseudo-label discovery, local anomaly neighbor embedding aggregate, and local discriminator module.

2) *Evaluation Metrics*: We adopt receiver operating characteristic-area under the curve (ROC-AUC) as an evaluation metric to evaluate the performance of our proposed framework. ROC-AUC is commonly used as an evaluation metric for anomaly detection frameworks [7], [8]. The ROC curve consists of a false positive rate (FPR) and a true positive rate (TPR). The ground-truth labels and the results of the anomaly detection are used to calculate TPR and FPR. The closer the AUC is to 1, the better the performance of the method.

3) *Parameter Settings*: We set the number of nodes in the sampling graph as 4. We fixed the embedding dimension as 64. We fixed the batch size $B = 256$. We use the Adam to train the model. We set train epochs as 400 on the BlogCatalog and Flickr. We set train epochs as 100 on the Cora and Wiki. We set the learning rate as 0.001 on Cora, Wiki, and Flickr. We set the learning rate as 0.003 on the BlogCatalog. The learning rate for the local discriminator module in FedCAD is 0.05. For Cora, Wiki, BlogCatalog, and Flickr, the confidence threshold for determining the pseudo label is -0.3 , -0.5 , -0.5 , and -0.4 , respectively. For each dataset, we sample 256 rounds during the inference phase to obtain accurate detection results.

C. Anomaly Detection Results

The graph data of each client is obtained using random sampling results of experimental datasets with various scales to imitate the graph data distribution in the actual world. We set up four clients whose data proportions are [50%, 60%, 70%, 80%], respectively. We repeated the experiment five times to ensure that the framework is authentic and reliable.

The ROC curves of the client containing 50%, 60%, 70%, and 80% of the data size are shown in Figs. 6–9, respectively. Table II displays the AUC results for the four benchmark datasets for comparison by computing the area under the ROC curves. FedCAD significantly outperformed FedAD and also extremely surpassed other baselines. For the experimental results, we conducted the following analysis.

FedCAD performs better than other baselines. There are two main reasons.

- 1) Federated learning cooperates with multiple clients to learn together, potentially increasing data size for model training. An anomaly detection model learned from cooperative learning has better performance than a model trained using only the data of each client alone.
- 2) The proposed local anomaly node embedding update can expand the difference between abnormal nodes and their neighbors. It improves the contrastive learning performance and further improves the FedCAD performance.

Compared with FedAD, FedCAD first uses pseudo-label discovery to infer abnormal nodes. Then use pseudo labels to perform local anomaly node embedding updates to expand the difference between abnormal nodes and their neighbors, improve the quality of the instance pairs, and obtain a high-quality anomaly detection model.

D. Parameter Study

In this section, we explore the effect of three critical parameters on the performance of the proposed framework: the data size of clients, the number of clients, and the confidence threshold. We perform these experiments on the four datasets.

1) *Data Size of Client*: In this experiment, our goal is to explore the effect of data sizes on FedCAD. While keeping all other experimental parameters constant, we investigated two different scales of client configurations. Initially, we explored a setup involving three clients. Within this scenario, we designed three sets of experimental schemes. In these experiments, the proportion of data nodes held by each client (referred to as the sampling ratio) was specifically set as follows: the first group utilized [20%, 30%, 40%], the second group employed [20%, 40%, 60%], and the third group used [30%, 50%, 70%]. Subsequently, we increased the number of clients to five, ensuring that all other conditions unrelated to the sampling rate remained unchanged. We assigned a different sampling ratio for these five clients as [20%, 40%, 50%, 70%, 90%]. The purpose of these adjustments was to systematically observe and analyze the impact on the overall performance of changes in the number of clients and their respective proportions of data nodes. The experimental results are shown in Table III. For the experimental results, we conducted the following analysis.

- 1) It can be observed from [20%, 30%, 40%] that FedCAD can still achieve the best results even if the data size of each client is small. This is mainly attributed to that the local anomaly neighbor embedding aggregate module in FedCAD can enlarge the difference between the anomalous target node and its neighbors.
- 2) FedCAD still performs better than FedAD in most cases, especially below [20%, 40%, 50%, 70%, 90%], which

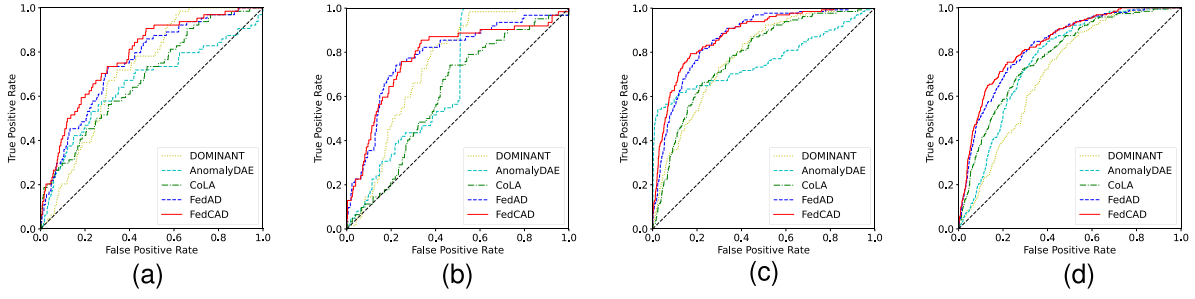


Fig. 6. ROC curves comparison for clients with 50% data volume on four benchmark datasets. (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

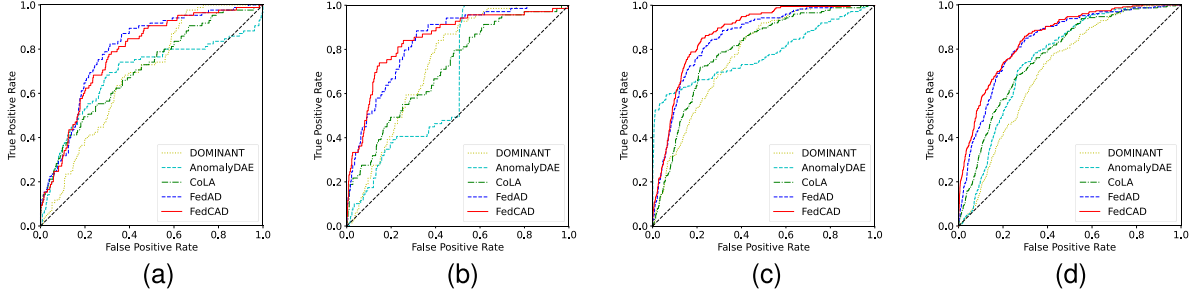


Fig. 7. ROC curves comparison for clients with 60% data volume on four benchmark datasets. (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

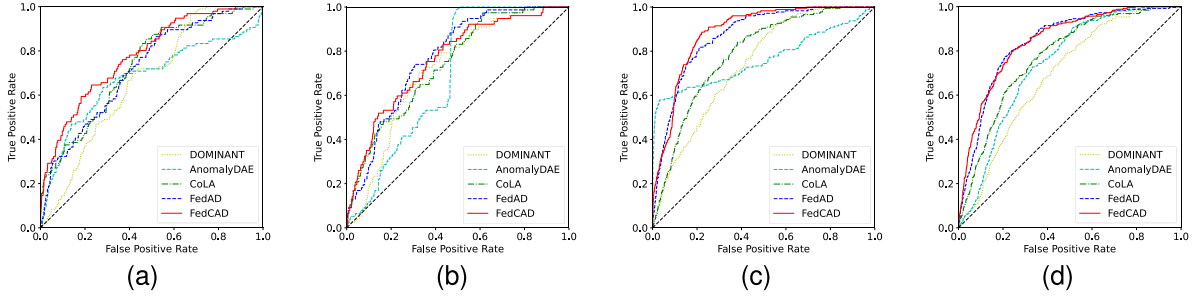


Fig. 8. ROC curves comparison for clients with 70% data volume on four benchmark datasets. (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

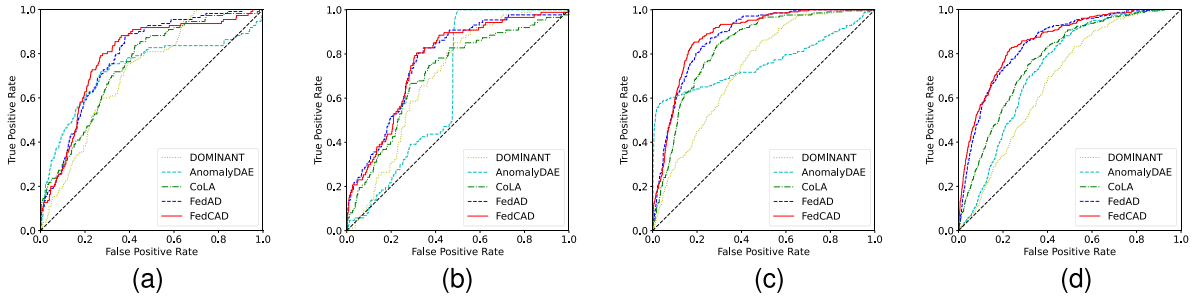


Fig. 9. ROC curves comparison for clients with 80% data volume on four benchmark datasets. (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

indicates that the proposed local anomaly neighbor embedding aggregate effectively enlarges the difference between anomaly nodes and their neighbors, and learns a high-quality FedCAD model.

3) Compared with Table II, FedCAD can still achieve good results and maintain its advantages despite the different number of clients and data size.

2) *Confidence Threshold*: Next, we study the impact of confidence threshold λ on our proposed FedCAD framework. The confidence threshold λ in (13) is used to control the number of pseudo labels. To study how λ affects the performance of FedCAD, we tune $\lambda \in [-0.1, -0.9]$ with step size 0.1. The results are shown in Fig. 10. As can be seen from the result,

choosing a suitable λ value can continuously improve the final AUC value. From the results, $[-0.3, -0.5]$ is a desirable interval.

3) *Number of Clients*: In the FedCAD framework, the number of clients is a crucial parameter significantly affecting performance and scalability. In this experiment, we aim to investigate the impact of the number of clients on FedCAD. We evaluate the performance of FedCAD on four datasets, keeping other parameters constant while varying the number of clients from 4 to 20. Notably, we also include FedAD as a baseline method for comparative analysis. As illustrated in Fig. 11, the results show that FedCAD consistently demonstrates substantial performance improvements over FedAD

TABLE II
AUC VALUES COMPARISON ON FOUR BENCHMARK DATASETS AT FOUR CLIENTS

Methods	Cora				Wiki				BlogCatalog				Flickr			
	50%	60%	70%	80%	50%	60%	70%	80%	50%	60%	70%	80%	50%	60%	70%	80%
DOMINANT	0.71607	0.68316	0.66475	0.71384	0.73879	0.73661	0.71674	0.69585	0.78240	0.77448	0.73948	0.73437	0.70821	0.70696	0.69080	0.67922
AnomalyDAE	0.65540	0.68445	0.67537	0.73010	0.67009	0.66244	0.68679	0.65245	0.75951	0.77651	0.76169	0.75786	0.76334	0.75708	0.74054	0.73326
CoLA	0.68370	0.70456	0.73044	0.74436	0.61127	0.71294	0.73856	0.70811	0.78018	0.79275	0.78640	0.83212	0.77757	0.77003	0.77201	0.77537
FedAD	0.74023	0.77274	0.74192	0.76799	0.77589	0.76883	0.76190	0.76869	0.83094	0.84042	0.83757	0.84337	0.72765	0.82296	0.82365	0.82295
FedCAD	0.77430	0.76942	0.76400	0.78422	0.77888	0.77804	0.75984	0.77336	0.85113	0.85485	0.85523	0.85725	0.79486	0.84067	0.83881	0.83853

TABLE III
AUC VALUES COMPARISON ON FOUR BENCHMARK DATASETS UNDER THE DIFFERENT NUMBERS OF CLIENTS AND DATA SIZE

Dataset	Method	20%	30%	40%	20%	40%	60%	30%	50%	70%	20%	40%	50%	70%	90%
Cora	CoLA	0.62480	0.70957	0.65681	0.73190	0.67772	0.72900	0.67628	0.68429	0.74039	0.69974	0.71005	0.72401	0.68347	0.70817
	FedAD	0.66732	0.70192	0.68762	0.78954	0.72397	0.76616	0.76567	0.75047	0.77962	0.69625	0.77870	0.77204	0.77823	0.73430
	FedCAD	0.67452	0.76425	0.73189	0.79094	0.72043	0.78699	0.76675	0.74269	0.78345	0.71930	0.75622	0.78674	0.78364	0.73862
Wiki	CoLA	0.46213	0.59593	0.63607	0.57411	0.61788	0.63241	0.65350	0.62553	0.67022	0.58400	0.64583	0.62406	0.68962	0.76331
	FedAD	0.48462	0.74514	0.75957	0.75346	0.62272	0.72572	0.70703	0.78464	0.75985	0.71957	0.75800	0.72953	0.75509	0.80220
	FedCAD	0.56483	0.74049	0.77498	0.71206	0.69179	0.76346	0.73704	0.77821	0.78436	0.72599	0.73448	0.75507	0.76771	0.82269
BlogCatalog	CoLA	0.62480	0.70957	0.65681	0.56135	0.64343	0.73710	0.54273	0.75909	0.76310	0.61978	0.72164	0.71688	0.74915	0.82274
	FedAD	0.64440	0.72940	0.68762	0.78564	0.79966	0.82600	0.77286	0.83179	0.81426	0.82774	0.86179	0.86622	0.85100	0.86077
	FedCAD	0.67452	0.76425	0.73189	0.81843	0.83723	0.82810	0.80327	0.84209	0.84274	0.88449	0.88617	0.88834	0.87101	0.88475
Flickr	CoLA	0.63428	0.59417	0.60962	0.69077	0.72971	0.76573	0.73179	0.73873	0.74367	0.71671	0.71414	0.70397	0.74289	0.74243
	FedAD	0.66957	0.75226	0.76741	0.71198	0.80947	0.81500	0.77764	0.79835	0.82792	0.73981	0.79835	0.80453	0.83369	0.83081
	FedCAD	0.73793	0.79636	0.79768	0.74517	0.82161	0.82887	0.81285	0.82307	0.83551	0.79431	0.81925	0.80870	0.85211	0.84072

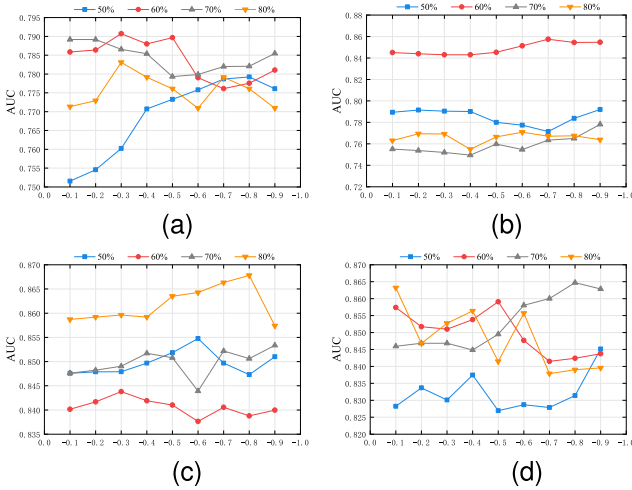


Fig. 10. AUC value under different confidence threshold λ . (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

on all datasets, demonstrating robustness to variations in the number of clients. However, as the number of clients reaches higher values, there is some degradation in the average performance between different clients, observed in both FedCAD and FedAD. This trend primarily arises due to the growing disparity in graph data distribution across clients, leading to more pronounced non-IID challenges. Nevertheless, FedCAD continues to exhibit relatively more minor performance fluctuations than FedAD, effectively demonstrating the scalability of the proposed FedCAD method.

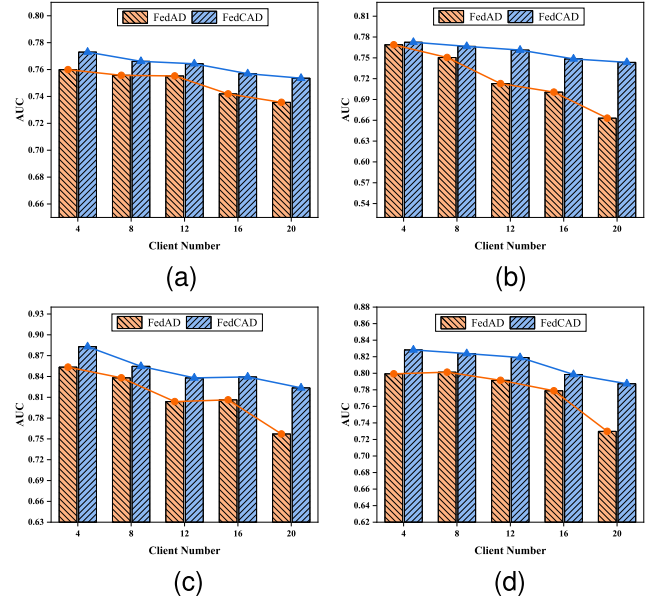


Fig. 11. AUC value under different client number. (a) Cora. (b) Wiki. (c) BlogCatalog. (d) Flickr.

VI. CONCLUSION

In this article, we design FedCAD for the task of distributed graph anomaly detection. First, FedCAD utilizes federated learning and does not require centralized client data for training. Furthermore, the sampling neighbor sub-graph of the target node obtained by contrastive learning is embedded in the low-dimensional embedding vector before

TABLE IV
AUC VALUES COMPARISON FOR GLOBAL GRAPH ANOMALY DETECTION ON FOUR BENCHMARK DATASETS

Dataset	DOMINANT	AnomalyDAE	CoLA	FedAD
Cora	0.76262	0.77376	0.74699	0.79652
Wiki	0.71183	0.74549	0.67515	0.74996
BlogCatalog	0.77519	0.79236	0.77862	0.80298
Flickr	0.73609	0.74869	0.76547	0.79935

being uploaded to the server. Second, FedCAD proposes a client-side abnormal neighbor aggregation mechanism based on contrastive learning. Specifically, each client aggregates the sampled neighbor subgraph embedding vectors of the other clients' abnormal nodes on average, improving local anomaly detection performance. This mechanism achieves the effect of data enhancement and solves the challenge of the data scale of each client being small. Experimental results demonstrate that FedCAD outperforms a range of baseline methods.

Despite FedCAD exhibiting superior performance, many areas can still be improved. The first area that can be improved is additional communication costs. The communication costs are high due to frequent parameter transfers between the federated learning client and server. In FedCAD, the transfer of vector embedding also increases communication costs. The second area that can be improved is to apply FedCAD to heterogeneous graphs. FedCAD can only be applied to the homogeneous graph. Due to the different types of nodes and links in the heterogeneous graph, the relationships between the nodes have various meanings. Therefore, FedCAD cannot be directly applied to heterogeneous graph. Future work can focus on appropriately reducing communication costs and applying it to the heterogeneous graph.

APPENDIX GLOBAL GRAPH ANOMALY DETECTION WITH FEDERATED LEARNING

We performed the global graph anomaly detection with federated learning on the server and obtained the results of the model trained on the global graph. Note that it is often very difficult to gather data together in the real world due to privacy security and industry competition. It is designed specifically for the academic experiment to test the performance of global graph anomaly detection with federated learning. The global graph data is collected from each client. Global graph anomaly detection with federated learning can approach or even exceed the performance of centralized training models by collecting data from clients.

Table IV displays the AUC results for the four benchmark datasets for comparison by computing the area under the ROC curve. Our proposed FedAD achieves the best anomaly detection performance in all four datasets. The main reason is that due to the particularity of graph data, each client's graph data can be regarded as a sample of the larger graph data, so there are overlapping nodes between clients. Each client uses the sampled graph data to train a local model, which amounts to data augmentation during training and therefore outperforms the baseline using a single graph data.

REFERENCES

- [1] G. Li and J. J. Jung, "Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges," *Inf. Fusion*, vol. 91, pp. 93–102, Mar. 2023.
- [2] C. Xiao, X. Xu, Y. Lei, K. Zhang, S. Liu, and F. Zhou, "Counterfactual graph learning for anomaly detection on attributed networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10540–10553, Oct. 2023.
- [3] A. D. Pazho, G. A. Noghre, A. A. Purkayastha, J. Vempati, O. Martin, and H. Tabkhi, "A survey of graph-based deep learning for anomaly detection in distributed systems," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 1, pp. 1–20, Jan. 2023.
- [4] R. Liu, P. Xing, Z. Deng, A. Li, C. Guan, and H. Yu, "Federated graph neural networks: Overview, techniques, and challenges," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, pp. 1–17, Feb. 2024.
- [5] B. Dong, D. Chen, Y. Wu, S. Tang, and Y. Zhuang, "FADngs: Federated learning for anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, pp. 1–15, Jan. 2024.
- [6] B. Perozzi and L. Akoglu, "Scalable anomaly ranking of attributed neighborhoods," in *Proc. SIAM Int. Conf. Data Min.*, 2016, pp. 207–215.
- [7] Z. Peng, M. Luo, J. Li, H. Liu, and Q. Zheng, "Anomalous: A joint modeling approach for anomaly detection on attributed networks," in *Proc. IJCAI*, 2018, pp. 3513–3519.
- [8] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 594–602.
- [9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [10] N. M. Foumani, L. Miller, C. W. Tan, G. I. Webb, G. Forestier, and M. Salehi, "Deep learning for time series classification and extrinsic regression: A current survey," *ACM Comput. Surv.*, vol. 56, no. 9, pp. 1–45, Oct. 2024.
- [11] H. T. Phan, N. T. Nguyen, and D. Hwang, "Aspect-level sentiment analysis: A survey of graph convolutional network methods," *Inf. Fusion*, vol. 91, pp. 149–172, Mar. 2023.
- [12] H. Fan, F. Zhang, and Z. Li, "Anomalydae: Dual autoencoder for anomaly detection on attributed networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2020, pp. 5685–5689.
- [13] Y. Li, X. Huang, J. Li, M. Du, and N. Zou, "SpecAE: Spectral autoencoder for anomaly detection in attributed networks," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2233–2236.
- [14] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "NetWalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 2672–2681.
- [15] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [16] B. Xu, J. Wang, Z. Zhao, H. Lin, and F. Xia, "Unsupervised anomaly detection on attributed networks with graph contrastive learning for consumer electronics security," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4062–4072, Feb. 2024.
- [17] P. Velicković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," 2018, *arXiv:1809.10341*.
- [18] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1150–1160.
- [19] Z. Peng et al., "Graph representation learning via graphical mutual information maximization," in *Proc. Web Conf.*, 2020, pp. 259–270.
- [20] X. Kong, W. Zhang, Y. Qu, X. Yao, and G. Shen, "FedAWR: An interactive federated active learning framework for air writing recognition," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 6423–6436, May 2024.

- [21] L. Wang, H. Zhou, Y. Bao, X. Yan, G. Shen, and X. Kong, "Horizontal federated recommender system: A survey," *ACM Comput. Surv.*, vol. 56, no. 9, pp. 1–42, Oct. 2024.
- [22] X. Kong et al., "A federated learning-based license plate recognition scheme for 5G-enabled Internet of Vehicles," *IEEE Trans. Ind. Informat.*, vol. 17, no. 12, pp. 8523–8530, Dec. 2021.
- [23] X. Kong, H. Gao, G. Shen, G. Duan, and S. K. Das, "FedVCP: A federated-learning-based cooperative positioning scheme for social Internet of Vehicles," *IEEE Trans. Computat. Social Syst.*, vol. 9, no. 1, pp. 197–206, Feb. 2022.
- [24] J. Du, N. Qin, D. Huang, Y. Zhang, and X. Jia, "An efficient federated learning framework for machinery fault diagnosis with improved model aggregation and local model training," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, pp. 1–24, Feb. 2023.
- [25] X. Wu, X. Yao, and C.-L. Wang, "FedSCR: Structure-based communication reduction for federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1565–1577, Jul. 2021.
- [26] Z. Wu et al., "FedICT: Federated multi-task distillation for multi-access edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 6, pp. 1107–1121, Jun. 2023.
- [27] H. Jamali-Rad, M. Abdizadeh, and A. Singh, "Federated learning with taskonomy for non-IID data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8719–8730, Nov. 2022.
- [28] D. Caldarola, M. Mancini, F. Galasso, M. Ciccone, E. Rodola, and B. Caputo, "Cluster-driven graph federated learning over multiple domains," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2021, pp. 2749–2758.
- [29] C. He et al., "FedGraphNN: A federated learning system and benchmark for graph neural networks," 2021, *arXiv:2104.07145*.
- [30] M. Jiang, T. Jung, R. Karl, and T. Zhao, "Federated dynamic GNN with secure aggregation," 2020, *arXiv:2009.07351*.
- [31] C. Chen et al., "Vertically federated graph neural network for privacy-preserving node classification," 2020, *arXiv:2005.11903*.
- [32] C. Wu, F. Wu, Y. Cao, Y. Huang, and X. Xie, "FedGNN: Federated graph neural network for privacy-preserving recommendation," 2021, *arXiv:2102.04925*.
- [33] B. Wang, A. Li, H. Li, and Y. Chen, "GraphFL: A federated learning framework for semi-supervised node classification on graphs," 2020, *arXiv:2012.04187*.
- [34] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph federated learning with missing neighbor generation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 6671–6682.
- [35] H. Xie, J. Ma, L. Xiong, and C. Yang, "Federated graph classification over non-IID graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 18839–18852.
- [36] H. Peng, H. Li, Y. Song, V. Zheng, and J. Li, "Differentially private federated knowledge graphs embedding," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manag.*, Oct. 2021, pp. 1416–1425.
- [37] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, 2017, pp. 1273–1282.



Xiangjie Kong (Senior Member, IEEE) received the B.Sc. and Ph.D. degrees from Zhejiang University, Hangzhou, China, in 2004 and 2009, respectively.

He was an Associate Professor with the School of Software, Dalian University of Technology, Dalian, China. He is currently a Professor with the College of Computer Science and Technology, Zhejiang University of Technology, Hangzhou. He has authored over 160 scientific papers in international journals and conferences. His research interests include mobile computing, network science, and data science.



Wenyi Zhang received the B.S. degree from the College of Science, Jiangxi University of Science and Technology, Ganzhou, China, in 2020. He is currently pursuing the M.E. degree in computer science and technology with the School of Computer Science, Zhejiang University of Technology, Hangzhou, China.

His research interests include gesture recognition, mobile edge computing, and federated learning.



Hui Wang received the B.Sc. degree in cyber engineering from the Shandong Jianzhu University, Jinan, China, in 2020. He is currently pursuing the master's degree with the School of Software, Dalian University of Technology, Dalian, China.

His current research interests include federated learning, graph neural networks, and edge computing.



Mingliang Hou received the B.Sc. degree from Dezhou University, Dezhou, China, in 2012, and the M.Sc. degree from Shandong University, Jinan, China, in 2016. He is currently pursuing the Ph.D. degree with the School of Software, Dalian University of Technology, Dalian, China.

His research interests include graph learning, city science, and social computing.



Xin Chen received the B.Sc. degree in information security from Harbin Engineering University, Harbin, China, in 2020. He is currently pursuing the master's degree with the School of Software, Dalian University of Technology, Dalian, China.

His research interests include spatiotemporal graph learning, urban science, and social computing.



Xiaoran Yan (Member, IEEE) received the B.S. degree in computer science from Zhejiang University, Hangzhou, China, in 2007, and the Ph.D. degree in computer science from The University of New Mexico, Albuquerque, NM, USA, in 2013.

From 2013 to 2015, he was a Postdoctoral Research Associate at the University of Southern California Information Sciences Institute, Marina del Rey, CA, USA. He was an Assistant Research Scientist at Indiana University Network Science Institute, Bloomington, IN, USA, from 2015 to 2021. Since

2021, he has been with the Research Institute of Artificial Intelligence, Zhejiang Lab, Hangzhou, as a Research Expert (PI). He has authored multiple journal and conference papers, and codirected NIH and IMLS grants. His research interests include network science, swarm intelligence, statistical machine learning, and their transdisciplinary applications in social, communications, and biomedical networks.

Dr. Yan served as a Program Committee Member for major organizations such as IEEE and Association for the Advancement of Artificial Intelligence (AAAI).



Sajal K. Das (Fellow, IEEE) is currently a Professor of computer science and the Daniel St. Clair Endowed Chair at Missouri University of Science and Technology, Rolla, MO, USA. He has coauthored four books and holds five U.S. patents. His research interests include wireless sensor networks, mobile and pervasive computing, UAVs, cyber-physical systems, the IoT, smart environments, distributed and cloud computing, cyber security, and social networks. He has made fundamental contributions to these areas and published extensively in these areas.

Dr. Das was a recipient of 12 best paper awards at prestigious conferences, IEEE Computer Society's Technical Achievement Award for pioneering contributions to sensor networks, and University of Missouri System President's Award for Sustained Career Excellence. His H-index is 96 with more than 37 100 citations according to Google Scholar. He is the Founding Editor-in-Chief of *Pervasive and Mobile Computing* journal, and an Associate Editor of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON MOBILE COMPUTING, ACM/IEEE TRANSACTIONS ON NETWORKING, and *ACM Transactions on Sensor Networks*.